

FLQuant — the building block

a4a Training Course — Module 1.2

Danai Mantopoulou Palouka & Alessandro Ligas

2026-09-13

The one class to understand

All data in FLR live inside an **FLQuant**.

- Catch numbers at age
- Weights at age
- Natural mortality, maturity
- Survey indices



Why we start here

FLQuant is not one more class to learn — it is **the** class. Everything we build for the rest of the week is made of these.

An FLQuant is always six-dimensional

Dimension	What it holds	Usual value
quant	age, length, ...	age
year	time	your year range
unit	sex or stock component	unique
season	within-year structure	all
area	spatial strata	unique
iter	iterations	1

Most of the time you only care about the **first two** and when we visit `simulate()` we will inspect the `iter`

Even an empty one has six dimensions

```
dim(FLQuant())
```

```
[1] 1 1 1 1 1 1
```

The first dimension is the flexible one — hence the name quant:

```
names(dimnames(FLQuant(dimnames = list(length = c(10, 20, 30)))))
```

```
[1] "length" "year"    "unit"    "season" "area"    "iter"
```

Creating one — from a vector

Values fill **down the quant first**, then across years.

```
FLQuant(1:6, dimnames = list(age = 1:3, year = 2000:2001), units = "kg")
```

An object of class "FLQuant"

```
, , unit = unique, season = all, area = unique
```

	year	
age	2000	2001
1	1	4
2	2	5
3	3	6

```
units:  kg
```

Creating one — other ways

```
FLQuant()                                # empty, 1x1x1x1x1x1
FLQuant(0.2, dimnames = list(age = 1:2)) # one value, recycled
FLQuant(matrix(1:6, nrow = 3), quant = "age") # from a matrix
FLQuant(array(rlnorm(100), dim = c(10, 5, 2))) # from an array
as.FLQuant(df)                             # from a data.frame
```



The usual route from your own data

`read.csv()` → `as.matrix()` → `FLQuant()`

That is exactly what we do in Module 1.5 with your files.

Data and metadata travel together

```
flq <- FLQuant(1:6,  
  dimnames = list(age = 1:3, year = 2000:2001),  
  units = "kg")  
dim(flq)  
  
[1] 3 2 1 1 1 1  
  
dimnames(flq)$year  
  
[1] "2000" "2001"  
  
units(flq)  
  
[1] "kg"
```

FLQuant() can handle (most of the times) units:

```
n <- FLQuant(1000, units = "1000")    # numbers, in thousands  
w <- FLQuant(0.5, units = "kg")       # mean weight
```

```
units(n * w)
```

```
[1] "t"
```

FLR does the unit algebra for you: thousands $\times$ kg = **tonnes**.



Which means

If your units are wrong, the arithmetic is still wrong — it just fails **quietly**. Set them when you build the object.

Two things that catch everybody

1. Dimnames are character, not numbers

```
dimnames(flq)$year
```

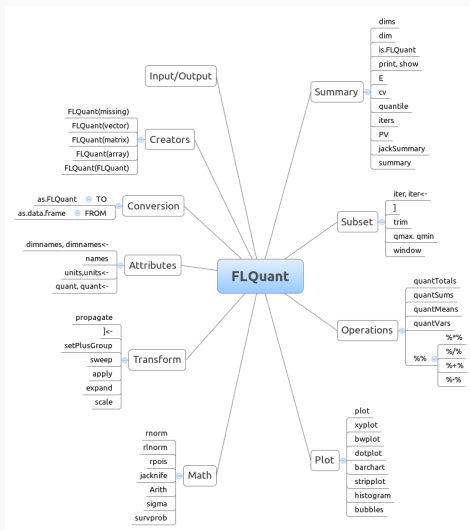
```
[1] "2000" "2001"
```

So subset by label, `flq[, "2000"]`, or by position, `flq[, 1]` — but `flq[, 2000]` is out of bounds.

2. Dimensions are not dropped by default

`flq[1, ]` is still a full 6D FLQuant. Use `drop = TRUE` to get a plain vector.

The FLQuant toolset



Summary

- One class, **six dimensions** — you mostly use the first two
- quant is flexible: age, length, whatever you need
- Carries **dimnames and units**, not just numbers
- Units are checked and combined by FLR — set them correctly
- Dimnames are **character**; dimensions are **not dropped**

Next: **Module 1.3** — FLStock, **built entirely out of FLQuants**.

Exercise

Open `handson_M1.2_flquant.R` and work through it.

1. Create `FLQuants` from a value, a vector and a matrix
2. Inspect `dim`, `dimnames` and `units`
3. Arithmetic: what conforms, what does not, and when to use `%*%`
4. Summaries: `quantSums`, `yearMeans`, geometric mean
5. Reproduce a target object from its printed output

Why six dimensions?

An FLQuant extends R's array, with a fixed dimension order:

`quant × year × unit × season × area × iter`

Fixing the order is what lets every FLR package agree on what it is being handed — the “common currency” from Module 1.1.

The 6th dimension, `iter`, stays at 1 all week until **Day 4**, where uncertainty is propagated through it.